Mission-Grade TT&C Communication System for CubeSats

Implementation Description

SATC-6 · Task IV · Document 2 of 4

1. Hardware Architecture

Both the space segment and the ground segment are built on the same core hardware pattern: an STM32F4-class 'Black Pill' microcontroller (LOGICAL_LEN protocol processing, SPI-connected to a Semtech SX1276-class LoRa transceiver module). The ground segment additionally bridges to an ESP32 module over UART, which exposes a USB-serial link to the operator's PC running the Python GUI.

Component	Space Segment	Ground Segment
MCU	STM32F4 Black Pill	STM32F4 Black Pill
RF module	SX1276-class LoRa, 433 MHz	SX1276-class LoRa, 433 MHz
RF interface	SPI (SS=PA4, RST=PB0, DIO0=PB1)	SPI (SS=PA4, RST=PB0, DIO0=PB1)
Actuator / sensor	PC13 LED (simulated spacecraft state)	n/a
Host bridge	USB-serial debug (Serial, 115200 baud)	HardwareSerial to ESP32 (PA2/PA3, 115200 baud)
Operator interface	n/a	ESP32 → USB → Python Tkinter GUI

2. Software Architecture and Module Structure

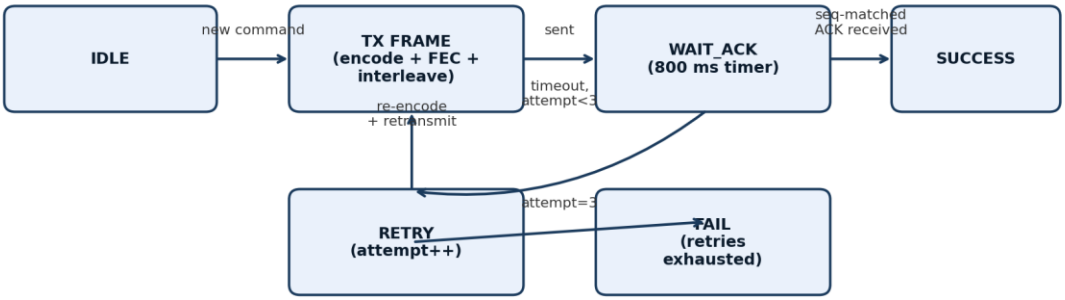
Both firmware images share an identical protocol-stack module structure, differing only in the command-dispatch logic (satellite executes commands; ground station originates them) and the presence of Doppler-compensation logic (ground only).

Module	Responsibility
crc16_ccitt()	16-bit CRC over the 7-byte pre-CRC portion of the logical frame (CCITT polynomial 0x1021)
computeMAC()	Keyed XOR checksum over src/dst/seq/cmd/data, providing lightweight command authentication
buildLogicalFrame() / validateLogicalFrame()	Assembles / verifies the 9-byte logical frame (sync, addressing, sequence, command, data, MAC, CRC)
hamming74Encode() / hamming74Decode()	Per-nibble Hamming(7,4) FEC encode and single-bit-error-correcting decode
interleave() / deinterleave()	3x6 block interleaver spreading burst errors across independent FEC codewords
encodeAndSend() / decodeReceived()	Full TX/RX pipeline: logical frame -> FEC -> interleave -> LoRa, and the inverse on receive
sendCommandWithRetry() (GCS only)	ARQ state machine: encode, transmit, wait for sequence-matched ACK/NACK, retry up to 3 times, report KPIs
isReplay() / recordSeq() (Satellite only)	8-entry anti-replay sliding window over received sequence numbers
computeDopplerShift() / applyDopplerCompensation() (GCS only)	Cosine-model Doppler prediction and LoRa.setFrequency() correction, gated by a PASS_START-triggered timer

3. State Machine: Command ARQ

The ground station's command dispatch follows a bounded-retry stop-and-wait state machine, illustrated below. A command is considered failed only after all retry attempts are exhausted without a sequence-matched ACK.

Figure 2 — GCS Command ARQ State Machine



4. Frame Structure

4.1 Logical Frame (pre-FEC, 9 bytes)

Offset	Field	Size	Description
0	SYNC	1 byte	Fixed synchronization byte, 0xAA
1	SRC	1 byte	Source address (1 = GCS, 2 = Satellite)
2	DST	1 byte	Destination address
3	SEQ	1 byte	Sequence number, incremented per new command; used for ACK matching and anti-replay
4	CMD	1 byte	Command identifier (see Section 5 of the User & Operations Manual)
5	DATA	1 byte	Single-byte command payload (e.g., blink count) or response payload (e.g., LED state)
6	MAC	1 byte	Keyed XOR authentication tag over bytes 1-5
7-8	CRC16	2 bytes	CRC16-CCITT over bytes 0-6, big-endian

4.2 Wire Frame (post-FEC, 18 bytes)

Each of the 9 logical bytes is split into two 4-bit nibbles. Each nibble is encoded independently with Hamming(7,4), producing 18 codeword bytes (each occupying the low 7 bits of a byte). The 18 codeword bytes are then passed through the 3x6 block interleaver before transmission. This is the actual byte sequence carried over the LoRa physical layer.

5. Resource Usage Analysis

The protocol stack (CRC, MAC, Hamming FEC tables computed inline rather than looked up, interleaver, ARQ state) is implemented without dynamic memory allocation — all buffers are fixed-size global arrays sized to LOGICAL_LEN (9) and ENCODED_LEN (18) bytes, keeping RAM usage small and deterministic, consistent with the ≤5 W CubeSat power budget target and typical STM32F4 SRAM constraints (commonly 64-192 KB depending on part).

Resource	Estimate	Basis
Static RAM (protocol buffers)	< 200 bytes	Six fixed arrays of 9-18 bytes each (TX/RX logical + encoded + FEC scratch)
Flash (protocol stack)	< 4 KB	Estimated from function count/complexity; to be confirmed from compiler .map output
Per-frame processing time	< 5 ms	18 Hamming encode/decode operations plus one CRC pass, well within the 800 ms ACK timeout budget
Space segment power draw	< 1 W active, radio-dominated	SX1276-class module TX current at +17 dBm is the dominant load; well within the 5 W budget

These figures are design-stage estimates. The Verification Report (Document 3) should be updated with measured flash/RAM usage taken directly from the STM32CubeIDE or Arduino IDE build output once the firmware is compiled against final board settings.

6. Summary

The firmware structure cleanly separates addressing/authentication (logical frame), reliability (FEC/interleaving/ARQ), and mission logic (command dispatch), which keeps the protocol stack testable in isolation — as demonstrated by the standalone verification of the Hamming(7,4) codec and interleaver described in the Verification Report.